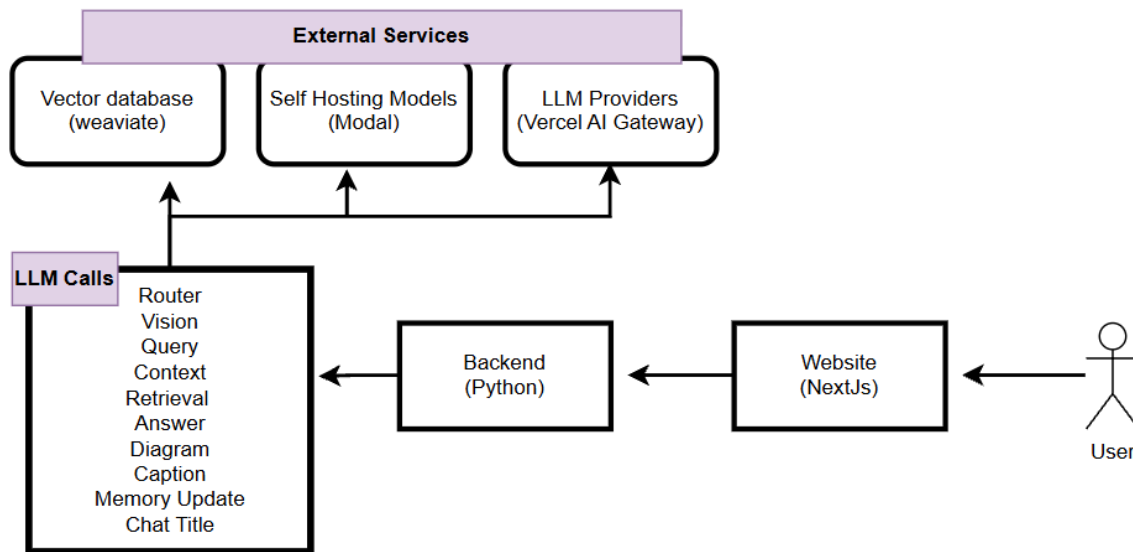


AIOPS for Networking - SLT Network Assistant

Live Deployed app: <https://skillful-nurturing-production-0306.up.railway.app/>

System Diagram



Response Generation Agentic System

1. Router
2. Vision
3. Context
4. Retrieval
5. Answer

1. Router

Reads each message and plans which steps it needs, so simple requests skip the steps they don't use.

- Example: "Draw 2 switches and 3 PCs" → only image generation, no search and no long answer.

2. Vision

Reads an attached topology image or screenshot into devices, models and cables.

- Example: a Packet Tracer screenshot → "Router0 (2911) connects to Switch0 (2960-24TT), which connects PC0 and PC1."

3. Context

In a long chat, pick only the earlier messages the new question depends on.

- Example: after talking about VLANs and then OSPF, "how do I check the neighbours?" → uses only the OSPF messages.

4. Retrieval

Knows how the knowledge base is organised and turns the question into a search with the right filters.

- Example: "Which Juniper access points work outdoors?" → Juniper, access points, Outdoor → AP61 and AP63.

5. Answer

Writes the reply from the question, the earlier messages, the image and the pages found, citing its sources.

- Example: "What is the PoE budget of a C9300X-48HX?" → the figure from the Catalyst 9300 Series page, marked [Source 1].

LLM Providers

Users can choose the provider and model for each agent in Settings. Hosted models have become inexpensive and are well suited to general knowledge such as computer networking. Where data security is required, open-weight models can run in our own deployment instead, so conversations are not sent to a model provider.

They are hosted on Modal, a platform for hosting AI models, which runs them serverless: GPUs start only when a request comes in, so we pay only while they run. Any new model deployed there appears in the app automatically.

- LLM Routing (Vercel AI Gateway)

MCP Server

Makes three features available to other AI apps (for example Claude Desktop) when someone needs them:

- Retrieval
- Image description
- Image generation

Fine Tuning

Most open source models already know networking well, so fine-tuning is not essential. If it is needed, there are several approaches. Each one learns from a different kind of dataset; one example row for each:

SFT – Learns from example questions with their correct answers.

```
{"prompt": "How do I create VLAN 20 named Staff on a Cisco switch?",  
"completion": "vlan 20\n name Staff"}
```

DPO – Learns from pairs of a better and a worse answer.

```
{"prompt": "Which command shows OSPF neighbours on a Cisco router?",  
"chosen": "show ip ospf neighbor",  
"rejected": "show ospf peers"}
```

KTO – Learns from single answers marked good or bad, like a thumbs up or down, so no pairs are needed.

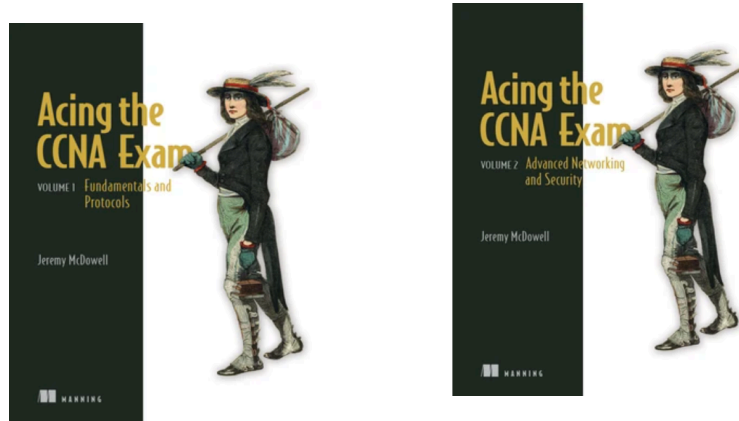
```
{"prompt": "How much power does PoE+ (802.3at) give per port?",  
"completion": "Up to 90 W per port.",  
"label": false}
```

ORPO (chosen) – Uses the same pairs as DPO, but learns the answers and the preference in one training stage, without a separate reference model, so training is simpler and cheaper.

```
{"prompt": "What is the default administrative distance of OSPF?",  
"chosen": "110",  
"rejected": "120"}
```

I have chosen **ORPO** over others due to its advantages.

Dataset Source:



1. Acing the CCNA Exam, Volume 1 - By Jeremy McDowell
2. Acing the CCNA Exam, Volume 2 - By Jeremy McDowell

Vector Database

142 device pages in Weaviate, searched with Qwen embeddings and a Qwen reranker.

CISCO (88)

Access points (49)

- Wi-Fi 7 · Indoor: CW9171, CW9172H, CW9172I, CW9174E, CW9174I, CW9176D1, CW9176I, CW9178I, CW9179F
- Wi-Fi 7 · Outdoor: CW9177D, CW9177E, CW9177I, CW9179F
- Wi-Fi 6E · Indoor: C9136I, CW9162I, CW9164I, CW9166, CW9166D1, MR57
- Wi-Fi 6E · Outdoor: CW9163E
- Wi-Fi 6E · Industrial: IW9165D, IW9165E, IW9167E, IW9167E-HZ, IW9167I
- Wi-Fi 6 · Indoor: C9105AXI, C9105AXW, C9115AXE, C9115AXI, C9120AXE, C9120AXI, C9130AXE, C9130AXI, MR28, MR36, MR36H, MR44, MR46, MR46E
- Wi-Fi 6 · Outdoor: C9124AXD, C9124AXE, C9124AXI, MR76, MR78, MR86
- Small business: Cisco Business 100, 150AX, 200, 240AC

Routers (23)

- Branch: Cisco 8100 Series, 8200 Series, 8300 Series, ISR 1000

- Meraki · Small branch: Cisco 8111-G2-MX, 8121-G2-MX, Meraki MX67, MX67C, MX67W, MX68, MX68CW, MX68W, MX75
- Meraki · Medium to large branch: Cisco 8355-G2-MX, MX85, MX95, MX105
- Meraki · Campus or concentrator: 8455-G2-MX, MX250, MX450
- Meraki · Multicloud: vMX Small, vMX Medium, vMX Large

Switches (16)

- Access: Catalyst 9200, Catalyst 9300 Series, Catalyst 9400 Series, C9350 Series
- Core and distribution: Catalyst 9500 Series, Catalyst 9600, C9550 Series, C9610 Series
- Data center and cloud: Nexus 3000 Series, Nexus 3550, Nexus 9000, N9300 Series, MDS 9000, Meraki MS120
- Small business and compact: Catalyst 9200CX, Meraki MS130

JUNIPER (54)

Access points (17)

- Indoor: AP12, AP21, AP24, AP32, AP33, AP34, AP36, AP37, AP41, AP43, AP45, AP47
- Indoor and outdoor: AP64, AP66
- Outdoor: AP61, AP63
- Other: BT11

Routers (24)

- MX Series: MX204, MX240, MX301, MX304, MX480, MX960, MX2008, MX2010, MX2020, MX10004 and MX10008
- ACX Series: ACX710, ACX5400, ACX7020, ACX7024 and ACX7024X, ACX7100, ACX7332 and ACX7348, ACX7509
- CTP Series: CTP150, CTP151, CTP2008, CTP2024, CTP2056
- PTX Series: PTX10000
- JRR Series: JRR200

Switches (13)

- QFX Series: QFX5110, QFX5120, QFX5130, QFX5200, QFX5210, QFX5220, QFX5230, QFX5240, QFX5700, QFX10002, QFX10008 and QFX10016
- EX Series: EX4650, EX5200

Context Management

Across chats (Mem0)

Without memory, users would have to type the same details again in every new chat. So when a message contains something useful for later, the assistant:

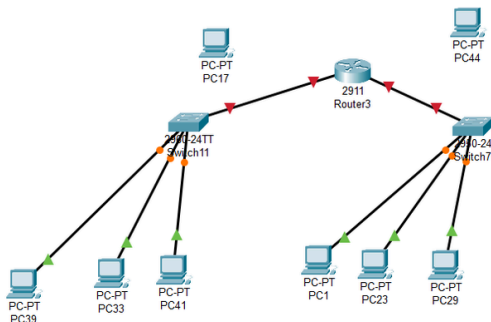
1. Identifies information that future chats may need
2. Checks whether it conflicts with a memory already saved
3. Resolves the conflict (the newest information wins) and saves it

Example: "My name is Manith" → useful later, so it is saved → retrieved in future chats when needed

Image Description Generation

When an image given to the vision language model, a predefined json structure should be output which contain network topology information

Input Image :



Description to be Generate:

```

{
  "devices": [
    {"name": "Router3", "type": "router", "model": "2911"},
    {"name": "Switch7", "type": "switch", "model": "2950-24"},
    {"name": "Switch11", "type": "switch", "model": "2960-24TT"},
    {"name": "PC1", "type": "pc", "model": "PC-PT"},
    {"name": "PC17", "type": "pc", "model": "PC-PT"},
    {"name": "PC23", "type": "pc", "model": "PC-PT"},
    {"name": "PC29", "type": "pc", "model": "PC-PT"},
    {"name": "PC33", "type": "pc", "model": "PC-PT"},
  ]
}

```

```

{"name": "PC39", "type": "pc", "model": "PC-PT"},
{"name": "PC41", "type": "pc", "model": "PC-PT"},
{"name": "PC44", "type": "pc", "model": "PC-PT"}
],
"links": [
  {"from": "Router3", "to": "Switch7"},
  {"from": "Router3", "to": "Switch11"},
  {"from": "Switch7", "to": "PC1"},
  {"from": "Switch7", "to": "PC23"},
  {"from": "Switch7", "to": "PC29"},
  {"from": "Switch11", "to": "PC33"},
  {"from": "Switch11", "to": "PC39"},
  {"from": "Switch11", "to": "PC41"}
]
}

```

Image Generation

Generative models like Flux, Stablediffusion is hard for precise network image generation. Therefore I used the graphviz library to arrange network components. The input will be the predefined json structure above mentioned.

Frontend

Developed on top of vercel chat template : <https://chatbot.ai-sdk.dev/demo>
Nextjs application

